

Statechart

Trentin Tommaso

December 26, 2025

Contents

1	Notazioni	2
1.1	Macchina a stati	2
1.2	Stato	2
1.3	Evento	2
1.4	Transizione di stato	2
1.5	Statechart diagram	2
1.5.1	Eventi con guardie mutualmente esclusive	2
1.5.2	Eventi che non scatenano transizioni	2
1.5.3	Stato finale / transizione	3
1.5.4	Esempio statechart diagram	3
2	Attività interne	3
2.1	Activity state	4
3	Superstati	4
3.1	Annidamento	4
3.2	Superstati	4
4	Stati concorrenti	5
5	Implementazione	5
5.1	Statechart da implementare	5
5.2	Implementazione switch nidificati	5
5.3	Implementazione pattern state	6
5.4	Implementazione tabelle di stato	6
6	Conclusioni - Applicabilità	6

1 Notazioni

1.1 Macchina a stati

Macchina caratterizzata da tre elementi fondamentali (stati, eventi, transizioni): la macchina può esistere in un numero finito di **stati** e, in risposta a **eventi**, eseguire **transizioni** tra gli stati.

1.2 Stato

Condizione della vita di un'entità durante la quale essa può:

- Soddisfare una condizione;
- Eseguire un'attività;
- Aspettare un evento.

In un dato istante l'entità si può trovare in uno e un solo stato, in un certo stato essa si comporterà in uno specifico modo in risposta a eventi specifici.

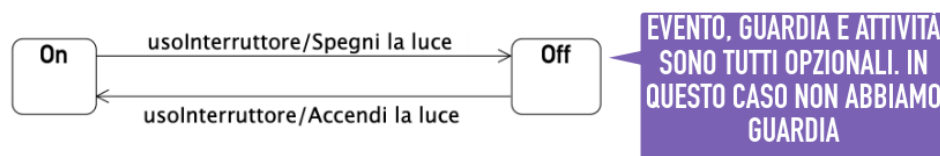
1.3 Evento

Occorrenza di interesse, collocata nel tempo e nello spazio, che può scatenare il cambiamento di stato dell'entità. Il comportamento della stessa entità di fronte agli eventi può cambiare se si trova in stati diversi.

1.4 Transizione di stato

Passaggio da uno stato all'altro, in risposta a un evento, etichettato con la sintassi **evento trigger[guardia]/attività**:

- **evento trigger** è l'evento (o sequenza di eventi) che scatena la transizione, può contenere parametri;
- **guardia** è una condizione booleana che deve essere verificata per il verificarsi della transizione;
- **attività** è un'azione elaborativa che si verifica contestualmente con la transizione.

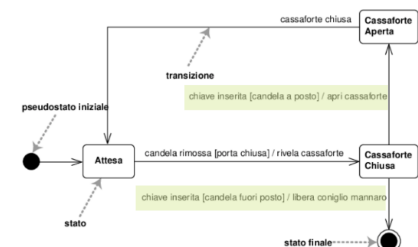


1.5 Statechart diagram

Grafo diretto in cui i nodi sono gli stati e gli archi sono le transizioni (complessivo di stati speciali quali **stato iniziale** e **stato finale**).

1.5.1 Eventi con guardie mutualmente esclusive

- Quando sistema si trova in uno stato, un evento può scatenare una sola transizione uscente da quello stato;
- Quando sistema si trova in uno stato, lo stesso evento trigger può portare a due stati diversi solo se guardie sono mutualmente esclusive.



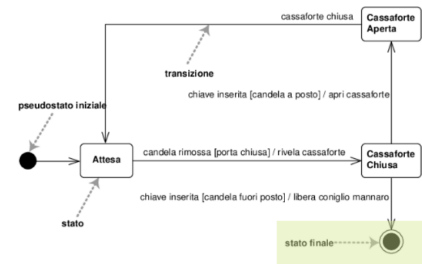
1.5.2 Eventi che non scatenano transizioni

- Se in uno stato si verifica un evento per cui non esistono transizioni valide, quell'evento viene ignorato;
- Es. se sistema si trova in stato Cassaforte chiusa, eventi di cassaforte chiusa e candela rimossa non causano transizioni.

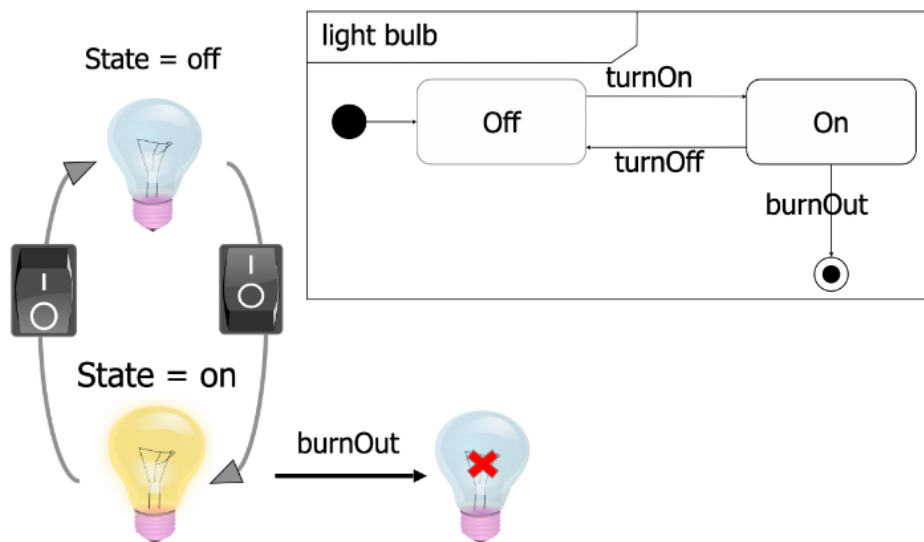


1.5.3 Stato finale / transizione

- Stato finale indica completamento dell'esecuzione della macchina a stati;
- Solitamente passaggio a stato finale sottintende distruzione dell'oggetto;
- Bisogna ricreare oggetto per ritornare allo stato iniziale.



1.5.4 Esempio statechart diagram



2 Attività interne

Risposte ad eventi che conducono a stesso stato possono essere indicate come **auto-transizioni** o **attività interne**, in queste attività interne uno stato risponde a eventi senza eseguire transizione a un'altro stato, ma scatenando attività interne (evento, guardia e attività vengono riportate nel box dello stato). La differenza tra *auto-transizioni* e *attività interne*

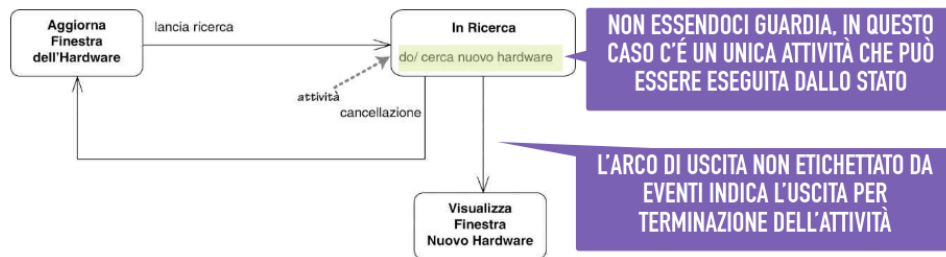


interne è che nelle ultime non avviene il nuovo ingresso nello stato (non vengono quindi rieseguite le attività di *entry* e *exit*).



2.1 Activity state

Stati in cui l'oggetto svolge determinata attività in modo continuato invece di essere in attesa inerte di un evento per agire, le **do-activity** sono attività interne particolari svolte da entità mentre si trova nello stato. Le do-activity



hanno durata temporale finita e possono essere interrotte, mentre le attività regolari sono istantanee ed atomiche, non influenzate / interrotte da altri eventi.

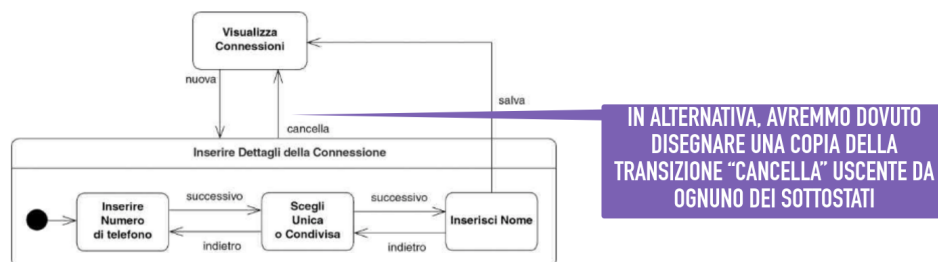
3 Superstati

3.1 Annidamento

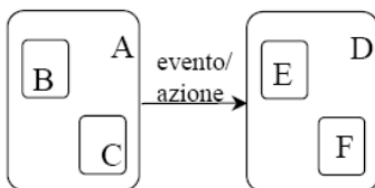
Strato composto: annidamento (*nesting*) di stati permette di dettagliare comportamento interno di uno stato mediante sottostati.

3.2 Superstati

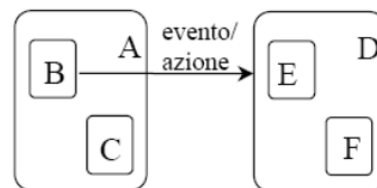
Stati composti che contengono un intero statechart diagram (gli stati del diagramma contenuto sono chiamati sottostati). I sottostati possono condividere certe transizioni / attività interne, mentre il superstato raccoglie il comportamento dei suoi sottostati. Quando gli archi sono diretti verso il superstato, in generale si intende che sono diretti



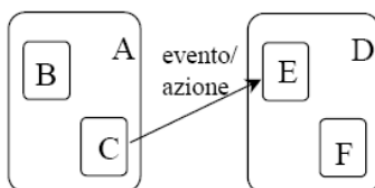
verso il so stato iniziale di default.



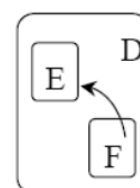
1) da un sottostato di A al sottostato di default di D



2) dal sottostato B al sottostato di default di D



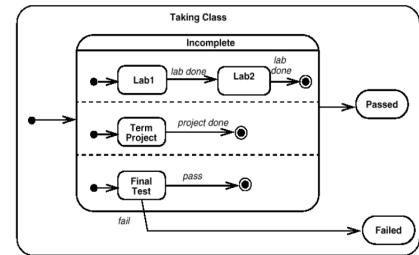
3) dal sottostato C al sottostato E



4) dal sottostato F al sottostato E

4 Stati concorrenti

Uno stato può essere diviso (mediante linee tratteggiate interne al superstato) in regioni che contengono sottostati da eseguire in modo concorrente.

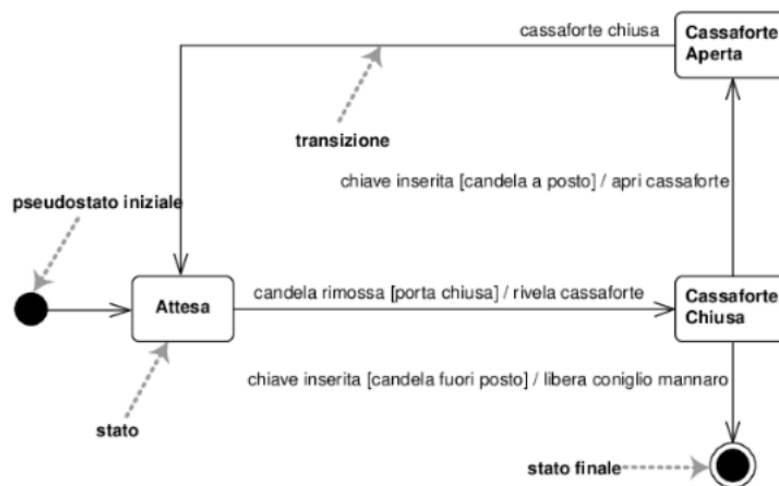


5 Implementazione

Uno statechart (a livello di dettaglio) può essere implementato in tre modi principali:

- utilizzando una catena di switch nidificati;
- applicando Pattern State;
- utilizzando tabelle di stato.

5.1 Statechart da implementare



5.2 Implementazione switch nidificati

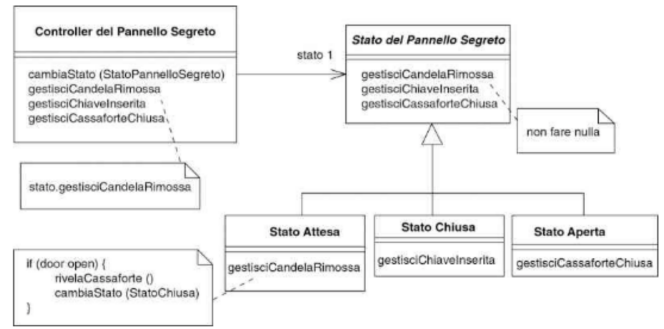
- **Vantaggi**
 - Soluzione più intuitiva.
- **Svantaggi**
 - Implementazione complessa (codice complesso anche per casi semplici);
 - Necessità di gestire modifiche apportate in più punti del codice.

```
public void GestisciEvento (EventoCassa unEvento) {
    switch (StatoCorrente) {
        case StatoCassa.Aperta :
            switch (unEvento) {
                case EventoCassa.CassaChiusa :
                    StatoCorrente = StatoCassa.Attesa;
                    break;
            }
            break;
        case StatoCassa.Attesa :
            switch (unEvento) {
                case EventoCassa.CandelaRimossa :
                    if (isPortaAperta) {
                        RivelaCassaforte();
                        StatoCorrente = StatoCassa.Chiusa;
                    }
                    break;
            }
            break;
        case StatoCassa.Chiusa :
            switch (unEvento) {
                case EventoCassa.ChiaveInserita :
                    if (isCandeIaAPosto) {
                        ApriCassa();
                        StatoCorrente = StatoCassa.Aperta;
                    } else {
                        LiberaConiglioMannaro();
                        StatoCorrente = StatoCassa.Finale;
                    }
                    break;
            }
            break;
    }
}
```

5.3 Implementazione pattern state

Statechart viene tradotto con:

- Classe **controller** che mantiene traccia dello stato corrente e implementa metodo di cambiamento di stato / si limita a inoltrare chiamate a classi stato;
- Classe **di stato** che dichiara metodi corrispondenti a ogni evento e li implementa in modo che non facciano nulla;
- Classi **stato derivate** corrispondenti agli stati del programma, implementano soltanto metodi corrispondenti a eventi scatenabili su quello stato per overriding.



5.4 Implementazione tabelle di stato

Rappresentazione tabellare delle informazioni contenute nel diagramma, tabella direttamente interpretata a runtime da apposito interprete oppure trasformata in codice da strumenti di generazione automatica del codice.

Stato sorgente	Stato destinazione	Evento	Guardia	Procedura
Attesa	Chiusa	Candela rimossa	Porta aperta	Rivela cassaforte
Chiusa	Aperta	Chiave inserita	Candela a posto	Apri la cassaforte
Chiusa	Finale	Chiave inserita	Candela fuori posto	Libera il coniglio mannaro
Aperta	Attesa	Chiusura della		cassaforte

6 Conclusioni - Applicabilità

- **Vantaggi**
 - Diagrammi di stato servono a descrivere comportamento di un oggetto in più casi d'uso;
 - Utili a rappresentare classi con logiche interne complesse / particolarmente interessanti.
- **Svantaggi**
 - Non molto utili a descrivere collaborazioni tra oggetti (per questo scopo sono preferibili Sequence / Activity diagram);
 - Loro implementazione corrisponde a codice ripetitivo, quindi si usano spesso generatori automatici di codice.